

PraxisFlow AI - Architektur & Phasenplan

Projekt: PraxisFlow AI

Auftraggeber: Praxisklinik Binningen AG

Version: 1.0.0

Datum: 31.08.2026

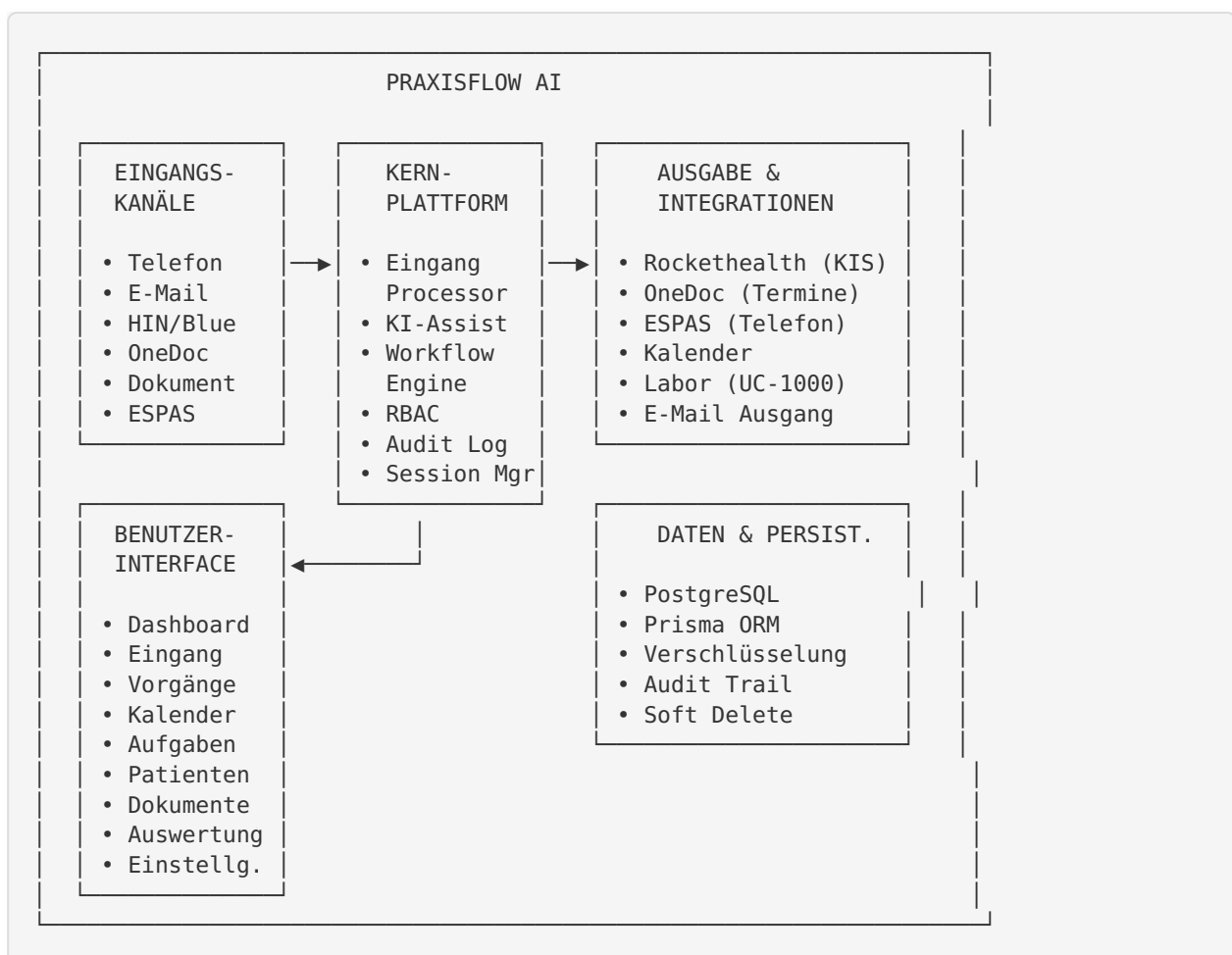
Status: Phase 1 – Architektur & Planung

1. Produktvision (Kurzfassung)

PraxisFlow AI ist die **zentrale digitale Rezeption und intelligente Verarbeitungsschicht** der Praxisklinik Binningen AG. Es ist kein EHR-Ersatz (kein Ersatz für Rockethealth), sondern eine strukturierte Eingangs- und Vorgangsverarbeitungsplattform.

Kernsatz: Eine eingehende Information wird automatisch erkannt und als sinnvoll vorbereiteter Vorgang dargestellt. Eine berechnete Mitarbeiterin prüft und bestätigt die vorgeschlagene Aktion.

2. Systemübersicht



3. Technischer Stack

Schicht	Technologie	Begründung
Frontend	Next.js (App Router), TypeScript, Tailwind CSS	Vollständig portabler SSR/SSG-Stack, breite Community
API	Next.js API Routes + tRPC	Typsichere end-to-end APIs ohne Codeduplizierung
Auth	Auth.js (NextAuth v5 stable)	Serverseite Session-Kontrolle, widerrufbar, RBAC-fähig
Datenbank	PostgreSQL 16	ACID-konform, bewährt, selbst hostbar
ORM	Prisma 5	Versionierte Migrationen, typsicher, exportierbares Schema
Tests	Vitest (Unit/Integration) + Playwright (E2E)	Open Source, portabel
CI/CD	GitHub Actions	Kein Vendor-Lock-in
Containerisierung	Docker + Docker Compose	Lokale Ausführbarkeit, Produktionsvorbereitung
Logging	Pino (strukturiert, JSON)	Kein PII in Logs
Fehler-Tracking	OpenTelemetry (selbst hostbar)	Kein PII an externe Dienste

Design-Entscheidung: Kein proprietärer Abacus-Lock-in

Die gesamte Geschäftslogik liegt in Standard-TypeScript-Modulen. Next.js läuft auf jedem Node.js-Host (Vercel, Hetzner, AWS, selbst gehostet). Die Datenbank ist ein Standard-PostgreSQL. Alle Secrets kommen aus Umgebungsvariablen.

4. Komponentenarchitektur

```

src/
├── app/
│   ├── (auth)/
│   │   └── # Next.js App Router
│   │   └── # Login, MFA, Session
│   ├── (app)/
│   │   └── # Geschützte Bereiche
│   │   ├── dashboard/
│   │   │   └── # Übersicht
│   │   ├── eingang/
│   │   │   └── # Eingänge verwalten
│   │   ├── kalender/
│   │   │   └── # Kalenderansicht
│   │   ├── vorgaenge/
│   │   │   └── # Vorgangsliste & Detail
│   │   ├── aufgaben/
│   │   │   └── # Aufgabenboard
│   │   ├── patienten/
│   │   │   └── # Patientenliste
│   │   ├── dokumente/
│   │   │   └── # Dokumentverwaltung
│   │   ├── auswertungen/
│   │   │   └── # Reports & Statistiken
│   │   └── einstellungen/
│   │       └── # Konfiguration
│   ├── api/
│   │   ├── # API-Endpunkte
│   │   ├── auth/
│   │   │   └── # Auth.js Handler
│   │   ├── trpc/
│   │   │   └── # tRPC Router
│   │   └── webhooks/
│   │       └── # Eingehende Webhooks
├── server/
│   ├── # Server-only Code
│   ├── auth/
│   │   └── # Session & RBAC
│   ├── db/
│   │   └── # Prisma Client & Queries
│   ├── services/
│   │   └── # Geschäftslogik
│   │   ├── eingang.service.ts
│   │   ├── vorgang.service.ts
│   │   ├── patient.service.ts
│   │   ├── aufgabe.service.ts
│   │   ├── freigabe.service.ts
│   │   └── audit.service.ts
│   ├── adapters/
│   │   ├── # Integration Mock-Adapter
│   │   ├── # Abstrakte Basis
│   │   ├── base.adapter.ts
│   │   ├── rockethealth/
│   │   ├── onedoc/
│   │   ├── espas/
│   │   ├── hin-blueconnect/
│   │   ├── email/
│   │   ├── kalender/
│   │   └── labor/
│   ├── ai/
│   │   ├── # KI-Assistenzfunktionen
│   │   ├── klassifizier.ts
│   │   │   └── # Kategorisierung
│   │   ├── patient-matcher.ts
│   │   │   └── # Patientenzuordnung
│   │   ├── summarizer.ts
│   │   │   └── # Zusammenfassungen
│   │   └── next-step.ts
│   │       └── # Nächste Schritte
├── lib/
│   ├── # Geteilte Utilities
│   ├── constants.ts
│   ├── date.ts
│   │   └── # CH-Datumsformat, Zürich TZ
│   ├── errors.ts
│   └── validators.ts
├── components/
│   ├── # React-Komponenten
│   ├── # Design System
│   ├── ui/
│   ├── layout/
│   ├── eingang/
│   ├── vorgang/
│   ├── patient/
│   └── shared/
└── prisma/
    ├── schema.prisma
    │   └── # Datenmodell
    ├── migrations/
    │   └── # Versionierte Migrationen
    └── seed.ts
        └── # Synthetische Testdaten

```

5. Phasenplan

Phase 1 - Architektur & Planung (AKTUELL)

Dauer: 1 Sprint

Ergebnis: Vollständige Dokumentation, kein Code ausser Schema-Entwürfen

- ☒ Systemarchitektur
- ☒ Komponentenübersicht
- ☒ Kerndatenmodell
- ☒ Rollen- und Rechtemodell
- ☒ Sicherheits- und Datenschutzkonzept
- ☒ ADR-Dokumente
- ☒ Repository-Grundstruktur
- ☒ Risiken & Annahmen

Phase 2 - Kern-App Pilot v1

Dauer: 3-4 Sprints

Voraussetzung: Freigabe Phase 1 durch Auftraggeber

- ☐ Datenbankmigrationen (Prisma)
- ☐ Auth.js Session-System mit RBAC
- ☐ API-Struktur (tRPC Router)
- ☐ Dashboard-Grundlayout
- ☐ Eingang-Verwaltung (vollständiger Workflow)
- ☐ Vorgangs-Engine
- ☐ Aufgabenboard
- ☐ Patientenliste & Zuordnung
- ☐ Synthetische Testdaten (Seed-Script)
- ☐ Mock-Adapter für alle 7 Integrationen
- ☐ KI-Assistenz (regelbasiert im Pilot)
- ☐ Audit-Log

Phase 3 - Qualität & Tests

Dauer: 2 Sprints

- ☐ Vitest Unit-Tests (Services, RBAC)
- ☐ Playwright E2E-Tests (Kernworkflows)
- ☐ Sicherheits-Audit (OWASP-Checkliste)
- ☐ Performance-Optimierung
- ☐ Barrierefreiheits-Check (WCAG 2.1 AA)
- ☐ Dokumentation abschliessen

Phase 4 - Pilot-Review & Freigabe

Dauer: 1 Sprint

- ☐ Pilot-Demo mit Praxisleitung
- ☐ Feedbackverarbeitung

- ☐ Abnahmekriterien prüfen
- ☐ Go-/No-Go-Entscheidung für Echtbetrieb

Phase 5 - Produktionsvorbereitung (Roadmap)

Dauer: TBD – nach Phase-4-Freigabe

- ☐ Echte API-Integrationen (nach Vertragsabschluss)
 - ☐ DSGVO/DSG-Rechtsgrundlagen dokumentiert
 - ☐ Penetrationstest (extern)
 - ☐ Schweizer Hosting-Evaluation (Exoscale CH / Infomaniak)
 - ☐ MFA-Aktivierung
 - ☐ Backup & Recovery-Test
 - ☐ Produktion
-

6. Annahmen (dokumentiert)

#	Annahme	Risiko	Massnahme
A01	PostgreSQL ist im Piloten ausreichend; kein Sharding notwendig	Niedrig	Schema mandantenfähig; Skalierung via Connection Pooling (PgBouncer)
A02	Alle Integrationen (Rockethealth, OneDoc etc.) sind Mock-Adapter	Hoch	Kein echter Datenaustausch im Pilot
A03	Auth.js v5 ist stabile API zum Bauzeitpunkt	Mittel	Abstraktion via eigene Auth-Schicht; ersetzbar
A04	KI-Funktionen im Pilot regelbasiert (kein LLM-Call)	Niedrig	LLM optional nachrüstbar via Adapter
A05	Pilotbetrieb ausschliesslich mit synthetischen Daten	Kritisch	Kein Produktivdatenimport; separate Seed-Scripts
A06	Eine Praxis (Mandant), ein Standort im Pilot	Niedrig	Mandanten-ID in allen Tabellen vorgesehen
A07	MFA vorbereitet, aber im Pilot noch nicht erzwungen	Mittel	TOTP-Interface implementiert, aber optional

7. Offene externe Abhängigkeiten

System	Status	Blockierung
Rockethealth API	✗ Keine API-Doku, keine Zugangsdaten	Ja – nur Mock im Pilot
OneDoc API	✗ Keine API-Doku, keine Zugangsdaten	Ja – nur Mock im Pilot
ESPAS API	✗ Keine API-Doku, keine Zugangsdaten	Ja – nur Mock im Pilot
HIN/BlueConnect	✗ Zertifikat & API fehlen	Ja – nur Mock im Pilot
Labor UC-1000/UC-1100	✗ Protokoll unbekannt (HL7? proprietär?)	Ja – nur Mock im Pilot
E-Mail (SMTP/IMAP)	⚠ Standard, aber Credentials fehlen	Teilweise – Mock mit echtem Protokoll
Kalender (CalDAV)	⚠ Standard, aber Endpunkt fehlt	Teilweise – Mock

Wichtig: Keine der obigen Integrationen wird als funktionsfähig bezeichnet, solange keine offizielle API-Dokumentation, vertragliche Freigabe und Zugangsdaten vorliegen.

8. Pilot-v1-Umfang (Scope-Festlegung)

Im Pilot enthalten

- 5 Benutzerrollen mit vollständiger RBAC
- Dashboard mit Ampelsystem
- Eingangs-Workflow (alle 6 Pilot-Workflows)
- Vorgangs-Engine mit Statusverwaltung
- Aufgabenboard
- Patientenliste mit Suchfunktion
- Dokumentliste
- Mock-Adapter für alle 7 Integrationspunkte
- Audit-Log (vollständig)
- Synthetische Testdaten (30 Patienten, 50 Eingänge)
- Responsive Design (Desktop, Tablet, Smartphone)
- Session-Sicherheit mit Widerruf
- Unit- und E2E-Tests

Nicht im Pilot (Phase 5+)

- Echte externe Integrationen

- MFA-Pflicht (vorbereitet, aber optional)
- Laborauswertungen (Rohdaten)
- Rechnungsstellung
- Mehrere Mandanten/Praxen
- Offline-Betrieb
- Mobile Native App

9. Deployment-Topologie (Pilot)

ABACUS CLOUD (Pilot)

Next.js App (Port 3000)

- └ SSR/API-Schicht
- └ Static Assets (Tailwind CSS)

PostgreSQL (Abacus-provisioniert)

- └ Synthetische Daten only

LOKAL (Docker Compose)

praxisflow-app:3000

- └ Next.js + TypeScript

praxisflow-db:5432

- └ PostgreSQL 16

praxisflow-adminer:8080

- └ DB-Admin-UI (Dev only)